

ALGORITMUL INVITATION

Alegerea liderului în sistemele distribuite

Eusebiu Rîmboi - Alexia Tanislav - Matei Ungureanu

Grupa 332

1 Introducere

Un sistem distribuit este o colecție de noduri autonome care cooperează pentru un scop comun.

Când un nod eșuează, sistemul trebuie să se adapteze. Există două strategii: operarea continuă (mascarea erorilor) sau oprirea temporară pentru reorganizare. Proiectul se concentrează pe a doua strategie: reorganizarea. Pentru a face reorganizarea, este nevoie de un coordonator, iar nodurile rămase active trebuie să cadă de acord asupra unui singur coordonator, având în vedere că nu au o memorie partajată și comunică doar prin mesaje.

Această documentație prezintă modalitatea în care algoritmul *Invitation* identifică corect noul lider după ce nodul anterior a eșuat.

2 Descrierea algoritmului serial

Algoritmul *Invitation* este o metodă de alegere a liderului în sistemele asincrone, care prioritizează fuzionarea grupurilor de noduri. Spre deosebire de algoritmi bazeți pe ID-uri statice, *Invitation* permite nodurilor să formeze grupuri conduse de un coordonator. Procesul fundamental constă în faptul că un coordonator de grup invită alți coordonatori să se alăture grupului său. Dacă invitația este acceptată, grupurile fuzionează, iar un nou identificator de grup este generat. Astfel, recuperarea după partiționarea rețelei este mai bună, permițând subsistemelor izolate să se reunească natural.

2.1 Grupul și starea nodului

Un grup este definit ca o mulțime de noduri care pot comunica între ele și care au acord asupra unui singur coordonator. Din punct de vedere logic, fiecare nod i menține un set de variabile de stare care definesc apartenența sa la un grup:

- **Starea (*status*):** Indică dacă nodul funcționează normal, se află în proces de alegere, reorganizează grupul sau este inactiv.

- **Identificatorul grupului (grp_i):** Un număr unic care identifică grupul curent din care face parte nodul. Acesta funcționează ca un „timestamp” logic pentru a distinge între epoci diferite ale aceluiași grup.
- **Coordonatorul (ldr_i sau $S(i).c$):** Identitatea nodului pe care i îl recunoaște drept lider.
- **Lista membrilor (Up):** Mulțimea nodurilor pe care coordonatorul le consideră active în grupul curent.

2.2 Fluxul logic al algoritmului

Logica serială a algoritmului poate fi descrisă printr-un ciclu de viață al grupurilor, care parcurge următoarele etape secvențiale:

1. **Inițializare și recuperare:** În momentul pornirii sistemului sau după detectarea unei defecțiuni majore (când un nod pierde contactul cu coordonatorul său), nodul intră într-o procedură de recuperare.
 - Nodul se declară singur într-un nou grup.
 - El devine automat coordonatorul acestui grup cu un singur membru.
 - Această stare asigură disponibilitatea imediată a serviciului, chiar și într-o formă degradată.
2. **Descoperirea:** Coordonatorii grupurilor trebuie să caute alți coordonatori activi în rețea pentru a extinde grupul. Periodic, un coordonator execută o procedură de verificare:
 - Coordonatorul trimite mesaje către alte noduri pentru a identifica dacă acestea sunt, la rândul lor, coordonatori ai altor grupuri.
 - Dacă sunt identificați alți coordonatori, algoritmul trece la etapa de fuziune.
3. **Fuziunea și invitația:** Pentru a evita haosul în care doi coordonatori încearcă simultan să preia controlul, se utilizează un mecanism de prioritate bazat pe ID-ul nodului.
 - Dacă un coordonator A descoperă un alt coordonator B , compară prioritățile. Dacă $Prioritate(A) > Prioritate(B)$, A inițiază procesul de fuziune.
 - Coordonatorul A trimite o invitație către B pentru a se alătura grupului său.
 - Spre deosebire de algoritmul Bully, unde nodurile sunt „forțate” să accepte liderul, aici nodul invitat (B) poate accepta invitația, aducând cu el toți membrii grupului său vechi în noul grup condus de A .
4. **Reorganizarea:** Invitația este acceptată:
 - Se generează un nou număr de grup (grp_{new}). Acesta este format, de regulă, din ID-ul noului coordonator concatenat cu un contor intern care se incrementează la fiecare fuziune.
 - Noul coordonator (A) centralizează lista tuturor membrilor (din grupul său și din grupurile invitate).
 - Se distribuie noua configurație (noul grp și noua listă de membri) către toate nodurile participante, iar sistemul revine în starea normală.

2.3 Gestionarea defectiunilor

Algoritmul tratează partiționarea rețelei ca o stare naturală. Dacă un grup mare se rupe din cauza unei erori de rețea, nodurile izolate detectează lipsa comunicării (timeout), acestea execută procedura de recuperare și formează grupuri noi, mai mici. Când conexiunea se restabilește, mecanismul de fuziune va reuni automat aceste grupuri într-unul singur.

2.4 Invitație refuzată

În cadrul algoritmului *Invitation*, un coordonator poate refuza o cerere de fuziune dacă aceasta încalcă criteriile de ordonare a sistemului (ex: dacă nodul invitat deține un identificador sau o prioritate superioară celei a inițiatorului). Acest refuz nu reprezintă un eșec al procedurii, ci un mecanism de rezolvare a conflictelor. Odată ce inițiatorul primește un refuz, acesta încetează din a invita grupul respectiv să se unească. Ca răspuns, nodul cu prioritate superioară (care a refuzat inițial) preia inițiativa și lansează o contra-invitație către fostul inițiator. Această logică de inversare a rolurilor asigură faptul că fuziunea grupurilor se realizează mereu sub coordonarea liderului cu cea mai mare prioritate, garantând astfel prevenirea apariției ciclurilor infinite de negociere.

2.5 Observație

Deși specificația originală Garcia-Molina presupune că nodurile conectate vor forma eventual un singur grup, analize ulterioare (precum cea a lui Stoller) arată că în rețele cu conectivitate non-tranzitivă (unde A vede B, B vede C, dar A nu vede C), logica serială simplă a invitației poate duce la formarea mai multor grupuri decât minimul necesar, deoarece un lider nu poate coordona noduri cu care nu are legătură directă. Însă, pentru scopul acestui algoritm, prioritatea este stabilitatea grupurilor formate și capacitatea de a funcționa în partiții.

3 Descrierea variantei distribuite a algoritmului în format SPMD

Algoritmul *Invitation* este structurat conform modelului SPMD (Single Program Multiple Data). Astfel, fiecare nod din sistem rulează exact același cod, comportamentul său diferit fiind determinat doar de starea sa internă (dacă este lider sau membru) și de mesajele primite.

3.1 Structuri de Date Locale

Fiecare nod i menține în memoria locală (și persistentă) următoarele variabile de stare:

- **State:** Starea curentă a nodului (*NORMAL*, *ELECTION*, *REORGANIZATION*, *DOWN*).
- **MyGroup:** Identificatorul unic al grupului curent.
- **Coordinator:** ID-ul nodului pe care i îl consideră coordonator.

- **Members:** Lista tuturor nodurilor din grupul curent (relevantă în special pentru coordonator).

3.2 Proceduri principale (pseudocod)

Algoritmul este condus de evenimente (mesaje sau timeout-uri).

3.2.1 Procedura de recuperare

Această procedură este apelată când un nod pornește sau când detectează că a fost izolat de grup. Nodul formează un grup nou cu el însuși.

```
PROCEDURE Recovery(i) IS
BEGIN
    // se creeaza un nou numar de grup unic
    MyGroup := NewGroupId(i);
    Coordinator := i;
    Members := {i};
    State := NORMAL;

    // porneste task-ul de verificare periodica
    StartTask(Check);
END PROCEDURE
```

3.2.2 Procedura de verificare

Doar coordonatorii rulează această procedură periodic pentru a găsi alți coordonatori și a începe fuziunea.

```
PROCEDURE Check(i) IS
BEGIN
    IF State == NORMAL AND Coordinator == i THEN
        // alege un set de noduri aleatoare sau vecini cunoscuti
        TargetNodes := SelectRandomNodes();

        FOR EACH node j IN TargetNodes DO
            SEND Message("AreYouCoordinator", MyGroup) TO j;
        END FOR
    END IF
END PROCEDURE
```

3.2.3 Gestionarea mesajului "AreYouCoordinator"

Când un nod primește această întrebare, răspunde doar dacă este el însuși un coordonator.

```
ON RECEIVE Message("AreYouCoordinator", sender_group) FROM j DO
BEGIN
    IF State == NORMAL AND Coordinator == i
        AND MyGroup != sender_group THEN
        SEND Message("IamCoordinator", MyGroup, Priority(i)) TO j;
```

```
END IF  
END
```

3.2.4 Procedura de fuziune și invitație

Dacă un coordonator primește confirmarea de la alt coordonator, inițiază procesul de fuziune.

```
PROCEDURE Merge(i, j) IS // i este initiatorul, j este invitatul  
BEGIN  
    // schimba starea ca sa blocheze alte operatii  
    State := ELECTION;  
  
    // creeaza un nou id de grup  
    NewGroup := GenerateNextGroup(MyGroup);  
  
    // trimite invitatia catre celalalt coordonator  
    SEND Message("Invitation", NewGroup) TO j;  
  
    // asteapta acceptul (cu timeout)  
    WAIT FOR Message("Accept", j) TIMEOUT T;  
  
    IF Timeout THEN  
        State := NORMAL; // Anuleaza fuziunea  
    ELSE  
        // reorganizarea = actualizeaza lista de membri  
        Members := Members UNION j.Members;  
  
        // anunta toti membrii despre noua configuratie  
        FOR EACH node k IN Members DO  
            SEND Message("NewConfiguration", NewGroup, Members) TO  
                k;  
        END FOR  
  
        State := NORMAL;  
    END IF  
END PROCEDURE
```

3.2.5 Acceptarea invitației

Când un coordonator primește o invitație, trebuie să își oprească propriul grup și să se alăture celui nou.

```
ON RECEIVE Message("Invitation", new_group_id) FROM leader DO  
BEGIN  
    IF State == NORMAL AND Coordinator == i THEN  
        State := REORGANIZATION;  
  
        // opreste activitatea in grupul curent  
        FOR EACH node k IN Members DO  
            SEND Message("StopProcessing") TO k;
```

```

    END FOR

    // accepta invitatia si trimite lista membrilor
    SEND Message("Accept", Members) TO leader;

    // actualizeaza starea locala
    Coordinator := leader;
    MyGroup := new_group_id;
    State := NORMAL;
END IF
END

```

3.3 Observații

Protocolul asigură următoarele:

1. **Atomicitatea:** Schimbările de stare (ex: trecerea de la un grup vechi la unul nou) se fac tranzacțional. Dacă un pas eșuează (timeout), nodurile revin la starea inițială sau declanșează recuperarea.
2. **Gestionarea timeout-urilor:** Orice mesaj trimis (invitație, verificare) are asociat un cronometru. Lipsa răspunsului nu blochează nodul la infinit.
3. **Prioritatea:** Pentru a evita ca două grupuri să se invite reciproc simultan, se folosește prioritatea. Grupul cu prioritate mai mare îl invită întotdeauna pe cel cu prioritate mai mică.

4 Analiză teoretică

Această secțiune analizează comportamentul algoritmului *Invitation* într-un mediu distribuit asincron, evaluând corectitudinea, performanța și limitările acestuia în raport cu ipotezele inițiale și cu topologiile de rețea.

4.1 Analiză corectitudine și premise

Spre deosebire de algoritmul *Bully*, care necesită un mediu sincron, algoritmul *Invitation* este conceput pentru un mediu „nesigur”.

4.1.1 Premise

Pentru funcționarea corectă, algoritmul necesită satisfacerea următoarelor premise, definite de Garcia-Molina (Ipotezele 1-7):

- **Stocare sigură:** Fiecare nod trebuie să dispună de o memorie persistentă care supraviețuiește restartării. Variabilele critice (numărul grupului, starea, id-ul coordonatorului) trebuie salvate atomic pentru a evita incoerența datelor la revenirea din *crash*.
- **Modelul de eșec:** Se presupune că nodurile se opresc complet în caz de eroare și nu transmit date eronate (nu există comportament bizantin).

- **Comunicare asincronă:** Nu se impune o limită superioară de timp pentru transmiterea mesajelor. dar se presupune că mesajele nu sunt generate spontan de rețea.

Nu se presupune fiabilitatea canalului de comunicație, iar partiționarea rețelei și pierderea mesajelor sunt tratate ca stări normale, nu ca excepții fatale.

4.1.2 Corectitudine

Conform lui Garcia-Molina, algoritmul garantează *asertiunea 3*: „*Toate nodurile dintr-un grup (care au același identificator de grup) sunt de acord asupra aceluiași coordonator*”. Aceasta previne situațiile de partiționare în interiorul unui grup format.

În ceea ce privește asertiunea 4 („*Un set de noduri care pot comunica între ele vor forma eventual un singur grup*”), analiza recentă realizată de Stoller în *Leader Election in Asynchronous Distributed Systems* demonstrează că aceasta nu este întotdeauna validă în rețele cu conectivitate non-tranzitivă. Dacă nodul A vede B, și B vede C, dar A nu vede C, algoritmul original poate oscila sau poate eșua în a uni toate cele trei noduri într-un singur grup stabil, deoarece coordonatorul nu poate menține legătura cu nodurile nedirecte.

4.2 Analiză complexitate timp/mesaj

Performanța algoritmului variază între monitorizare și starea de reorganizare (fuziune).

4.2.1 Complexitate mesaj

Fie N numărul de noduri dintr-un grup G_1 și M numărul de noduri dintr-un grup G_2 , unde G_1 invită pe G_2 . Procesul de fuziune implică:

1. **Inițiere:** 1 mesaj de tip invitație de la coordonatorul G_1 la coordonatorul G_2 .
2. **Oprire:** M mesaje de oprire a procesării (în grupul invitat).
3. **Acceptare:** 1 mesaj de acceptare (cu lista membrilor).
4. **Configurare:** $N + M$ mesaje trimise de noul lider către toți membrii reuniți.

Astfel, complexitatea mesajelor pentru o fuziune este de ordinul $\mathcal{O}(N+M)$. În cel mai rău caz (formarea unui grup global din noduri singulare), complexitatea totală atinge $\mathcal{O}(V^2)$, unde V este numărul total de noduri din sistem.

4.2.2 Complexitate timp

Deoarece sistemul este asincron, nu se poate garanta o limită superioară de timp. Timpul de detecție a unui nou grup depinde de intervalul de timp între verificările periodice ale procedurii. Un interval mic reduce latența fuziunii, dar crește congestia rețelei. Timpul de reorganizare este dominat de cel mai lent nod care trebuie să confirme primirea noii configurații.

4.3 Avantaje și dezavantaje la variația numărului de noduri

Avantaje:

- **Funcționare autonomă în sub-grupuri:** În rețele foarte mari, algoritmul permite funcționarea în sub-grupuri. Dacă o rețea de 1000 de noduri se rupe, sistemul nu se blochează global (ca la un algoritm centralizat strict), ci continuă să funcționeze în grupuri independente.
- **Minimizarea congestiei globale:** Spre deosebire de algoritmi care fac broadcast către toate nodurile (precum Bully în faza de start), aici mesajele de coordonare sunt limitate inițial la nivelul grupului local.

Dezavantaje:

- **Overhead la noduri multe:** Pe măsură ce un grup devine foarte mare (ex: sute de noduri), sarcina coordonatorului de a trimite periodic mesaje de verificare către toți membrii devine un punct de congestie (bottleneck).
- **Instabilitate la margine:** Dacă numărul de noduri este mare și rețeaua este instabilă, reorganizarea frecventă poate duce la o degradare severă a performanței (thrashing), unde sistemul petrece mai mult timp alegând lideri decât procesând sarcini utile.

4.4 Exemplificare pentru topologii specifice

4.4.1 Cazul 1: Topologie cu partiționare (split-brain)

Considerăm o rețea liniară $A - B - C - D$. Inițial, toate formează un grup cu coordonatorul D .

- **Scenariu:** Legătura dintre B și C se rupe.
- **Comportament:**
 1. Nodurile A și B nu mai primesc mesaje de la D . După timeout, rulează procedura de recuperare.
 2. A și B formează grupuri singleton. Apoi, prin procedura de verificare, B îl descoperă pe A (sau invers) și fuzionează.
 3. **Rezultat:** Sistemul se stabilizează în două grupuri independente: $\{A, B\}$ cu un lider local, și $\{C, D\}$ cu liderul vechi. Aceasta demonstrează robustețea algoritmului la partiționare.

4.4.2 Cazul 2: Conectivitate non-tranzitivă (scenariul propus de Stoller)

Considerăm o rețea liniară $A - B - C$.

- **Comportament:**
 1. Dacă B devine coordonator, poate forma un grup $\{A, B, C\}$.
 2. Totuși, dacă A devine coordonator, îl poate invita pe B . Dacă apoi B (acum membru simplu) îl "vede" pe C , nu poate iniția fuziunea.
 3. Dacă A încearcă să îl invite pe C (prin informații indirecte), invitația eșuează din lipsa legăturii directe.

5 Bibliografie

- Suport de curs: cursurile 2-8.
- Documentul aferent lucrării alese: *Elections in a Distributed Computing System.pdf*
- Lucrarea *Leader Election in Asynchronous Distributed Systems* - <https://www3.cs.stonybrook.edu/~stoller/paper/election-spec.pdf>
- Curs de la Universitatea din California San Diego - <https://cseweb.ucsd.edu/classes/sp16/cse291-e/applications/ln/lecture6.html>
- Curs de la Universitatea Carnegie Mellon - <https://www.andrew.cmu.edu/course/14-736-s19/applications/ln/14736-18.pdf>